

Monolingual Language Models for Hindi and Assamese: Data, Pretraining, and Symbolic Reasoning

Consolidated Final Report (Phases 1, 2 and 3)

Shubhadeep Mandal (Roll No. 2025201056)

Branch phase-3 • Language Models and Agents, Monsoon 2026

Higher-resource model H: Hindi (Devanagari) • Lower-resource model L: Assamese (Eastern Nagari, অসমীয়া)

Abstract. This report consolidates the whole project. In Phase 1 I built two strictly separate data pipelines and trained a 16,384-piece SentencePiece BPE tokenizer for each language; Hindi reached 723.32M cleaned tokens (20.55% collected manually) and Assamese 528.50M (22.48% manually). In Phase 2 I implemented and pretrained two ~ 25 M-parameter decoder-only transformers per language from scratch, a hand-written baseline (V1) and a modern variant (V2) with RoPE, SwiGLU and RMSNorm, on 500M tokens each. The modern model won everywhere: test perplexity 52.26 vs. 82.28 (Hindi) and 80.93 vs. 120.54 (Assamese). In Phase 3 I generated a leak-proof synthetic relational-reasoning suite and fine-tuned eight models (both architectures $\times$ direct SFT vs. chain-of-thought). Chain-of-thought training clearly improves output quality (answer F1, character similarity, decomposed step score), while direct SFT keeps a small edge in strict exact match. An attention comparison averaged over 200 test examples per language shows that fine-tuning leaves the early layers almost unchanged and reshapes the last two: answer tokens attend more sharply and place more of their attention on the premises. The final section answers the four comparison questions from the assignment brief.

Artifacts. Phase 1 corpora and tokenizers: <https://www.kaggle.com/datasets/shubhadeepmandal/lma-hindi-artifacts> and <https://www.kaggle.com/datasets/shubhadeepmandal/lma-assamese-artifact>; Phase 2 checkpoints: <https://www.kaggle.com/datasets/shubhadeepmandal/lma-phase2-artifacts>; Phase 3 checkpoints: <https://www.kaggle.com/datasets/shubhadeepmandal/lma-phase3-artifact>. Code: github.com/CL3-410/individual-project-Seronic2001 (branch phase-3).

1 Project overview

Phase 1, Data & tokenizers	Phase 2, Pretraining	Phase 3, Reasoning
Custom multi-threaded crawlers + textbook OCR; NFKC normalisation, script filtering, SHA-256 and MinHash dedup; 98/1/1 splits; four tokenizer candidates compared per language.	Two architectures per language, both 8 layers, $d_{\text{model}}=384$: V1 (learned positions, GELU, LayerNorm) and V2 (RoPE, SwiGLU, RMSNorm). 500M tokens, AdamW + cosine, fp16.	Synthetic comparative-reasoning suite (6 paradigms) with disjoint train/test entity pools; 8 fine-tuned models; 5-tier metric suite; pre- vs. post-finetune attention analysis.

The two languages share no data, code paths, tokenizers or parameters; every component exists twice, once per language. All models were trained on Kaggle GPUs (P100/T4) within the free tier.

2 Phase 1: corpora and tokenizers

2.1 Collecting and cleaning the data

For Hindi, large web corpora (Sangraha, OSCAR, Wikipedia) already exist, so the manual share came from my own crawlers over literature archives (Gadyakosh, KavitaKosh), news portals, and NCERT textbook PDFs (digital extraction plus Tesseract OCR). Assamese was harder. Public Assamese text is thin and often mislabelled as Bengali, so a larger fraction of the corpus came from targeted crawls over regional news portals (Asomiya Pratidin, NENow, Niyomiya Barta, EastMojo/ETV Bharat, Dainik Agradoot), literary archives, and OCR over 1,840 SCERT/SEBA/NCERT textbooks.

All documents from all sources passed through the same deterministic pipeline (`clean.py`):

- Unicode canonicalisation:** NFKC, zero-width character control (ZWJ/ZWNJ), nukta healing (e.g. ক + ় -> ক্), and repair of ASCII periods/pipes into the proper purna viram ।;

2. **script-ratio filter**: discard any document with under 80% of its characters in the target Unicode block (this removes English boilerplate and Bengali–Assamese confusion);
3. **deduplication**: exact SHA-256 hashes plus MinHash/LSH near-duplicate removal at a 0.6 Jaccard threshold (1,859,349 duplicates dropped in Hindi; 114,290 in Assamese);
4. **splitting**: deterministic 98/1/1 train/val/test split (seed 1337), then streamed into flat uint16 token binaries.

Table 1: Corpus summary (rubric: $\sim 500\text{M}$ tokens, $\geq 20\%$ manually collected; both languages comply).

	Hindi (H)	Assamese (L)
Total clean tokens	723,321,981	528,500,000
Manually collected	148,676,877 (20.55%)	118,800,000 (22.48%)
Downloaded corpora	574,645,104 (79.45%)	409,700,000 (77.52%)
Dedup removed	1,859,349 docs	114,290 docs

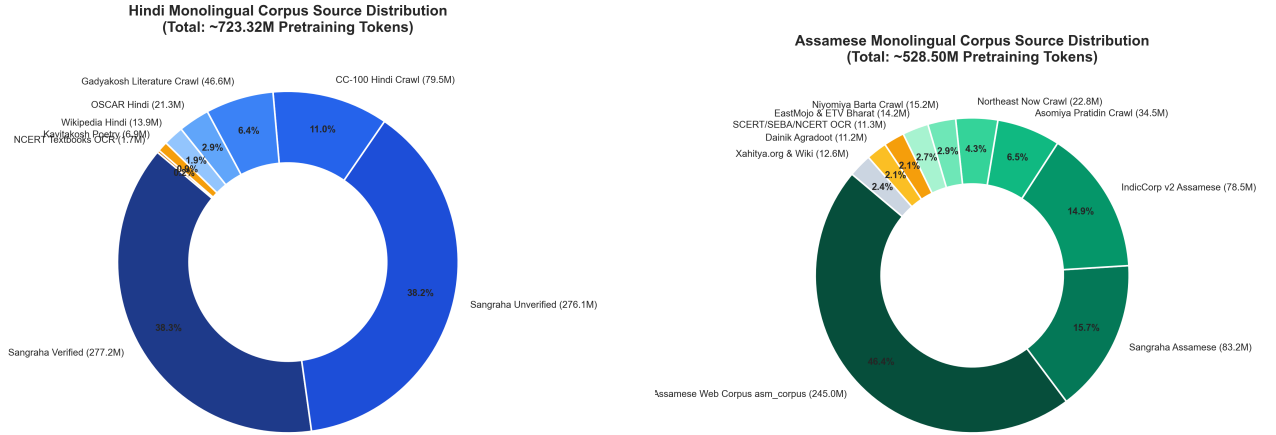


Figure 1: Corpus composition by source. Left: Hindi (723.32M tokens, 20.55% manual). Right: Assamese (528.50M tokens, 22.48% manual).

2.2 Choosing a tokenizer

Four candidates per language (BPE/Unigram $\times$ 16K/32K) were trained with SentencePiece (`character_coverage=1.0`, `byte_fallback=true`, digits split, scripts not merged) and scored on held-out text.

Table 2: Fertility (tokens per word) of the four candidates; unk rate was 0.0000% for all.

	BPE 16K	BPE 32K	Unigram 16K	Unigram 32K
Hindi	1.1858	1.1252	1.2728	1.2065
Assamese	1.4426	1.3130	1.4091	1.2978

The 32K vocabularies compress slightly better, but at $d_{\text{model}}=384$ a 32K embedding table costs $32,768 \times 384 = 12.58\text{M}$ parameters, half the model budget, while 16K costs 6.29M. Choosing 16K freed enough room to grow from 6 to 8 transformer layers, and the fertility penalty is small (Hindi 1.19 vs 1.13; Assamese 1.44 vs 1.31). Both chosen 16K BPE tokenizers reconstruct held-out text exactly (100% roundtrip) and never emit `<unk>` thanks to byte fallback.

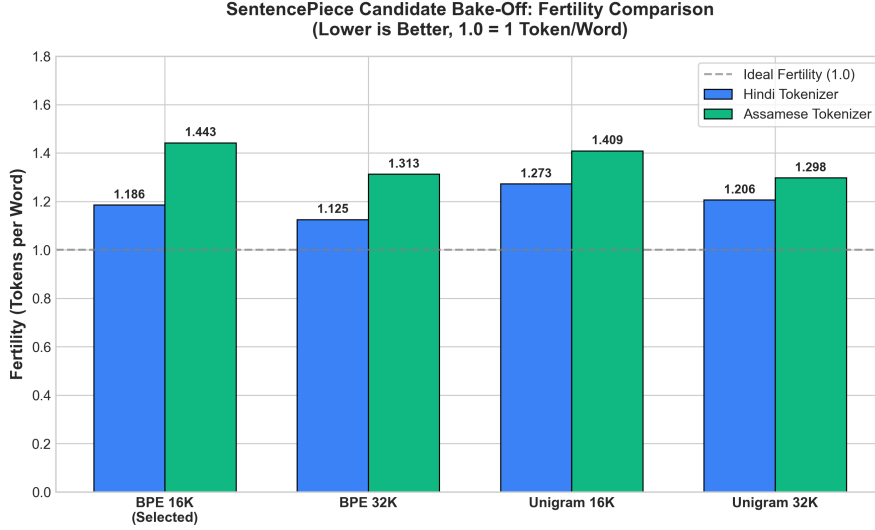


Figure 2: Fertility vs. vocabulary size for the four candidates.

3 Phase 2: pretraining two architectures per language

3.1 The two models

Both models share the same skeleton (decoder-only, 8 blocks, $d_{\text{model}}=384$, 6 heads, tied input/output embeddings) and differ in their internals:

- **Baseline V1** (24.98M params): learned absolute positional embeddings, hand-written causal attention (explicit upper-triangular mask), GELU MLP with $d_{\text{ff}}=2240$, pre-LayerNorm.
- **Modern V2** (25.17M params): RoPE rotary positions applied to Q/K, FlashAttention/SDPA kernel, SwiGLU MLP with $d_{\text{ff}}=1536$, pre-RMSNorm.

Both stay inside the 22.5–27.5M rubric window (6.29M embedding + 18.49M/18.88M blocks + norms). Causality was verified by perturbation: changing token $t+1$ leaves logits at positions $\leq t$ bit-identical in both architectures.

3.2 Pretraining and intrinsic results

Training used AdamW ($\beta_1=0.9$, $\beta_2=0.95$, wd 0.1), cosine decay from 6×10^{-4} with 40 warmup steps, micro-batch 32×16 accumulation (262,144 tokens per step), fp16 mixed precision, for 1,907 steps $\approx 500\text{M}$ tokens per model.

Table 3: Held-out test results (200 non-overlapping windows). $\text{PPL} = e^{\text{loss}}$; BPB is cross-entropy normalised by UTF-8 bytes.

Language	Model	Test loss (nats)	Test PPL	BPB
Hindi	Baseline V1	4.4102	82.28	0.5764
	Modern V2	3.9562	52.26	0.5189
Assamese	Baseline V1	4.7920	120.54	0.5641
	Modern V2	4.3935	80.93	0.5049

The architecture gap is nearly identical in both languages (-0.45 and -0.40 nats), so RoPE/SwiGLU/RMSNorm help regardless of script. The last column looks contradictory at first: Assamese has *worse* perplexity but *better* bits-per-byte. The higher token-level loss reflects the higher information carried per subword token (Section 5.2), not a worse model.

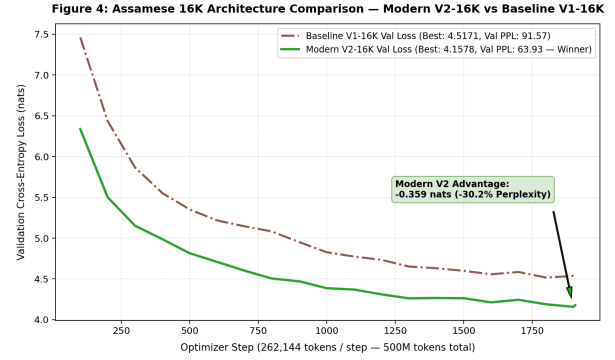
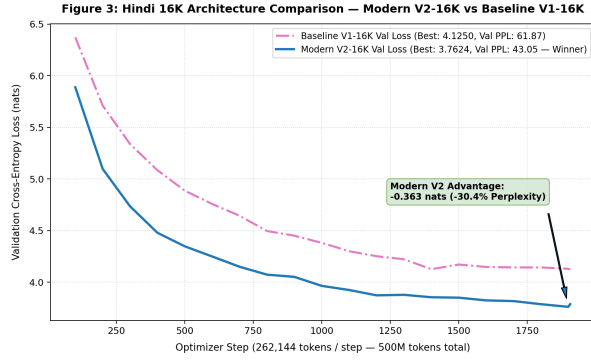


Figure 3: Validation loss over 1,907 steps: Modern V2 vs. Baseline V1. Left Hindi, right Assamese.

3.3 Generation quality

Generation was evaluated on 100 held-out prompts (32 tokens of context, 64 generated) at four temperatures. Greedy decoding collapses into repetition loops: rep-3 (share of duplicated 3-grams) hits 77.2% (Hindi) and 75.0% (Assamese). At $T=1.0$ the same models are fluent: rep-3 falls to 1.65% / 0.98%, distinct-2 reaches 88.3% / 96.6%, and chrF++ peaks at 19.86 (Hindi) and 24.02 (Assamese). At $T=1.5$ diversity keeps rising but syntax falls apart. This makes $T=1.0$ the sweet spot. One implementation detail matters here: stock ROUGE/chrF tooling tokenises with an ASCII regex and silently returns 0 on Indic scripts. `common/metrics.py` uses a Unicode-aware adapter; without it, every reference-based score would be a false zero.

4 Phase 3: direct SFT vs. chain-of-thought reasoning

4.1 Synthetic reasoning data

The assignment asks for synthetic comparative-reasoning data (“A is taller than B, B is taller than C...”). A minimal 2-premise template set is too easy for a 25M model: it can score $\geq 85\%$ by always copying the entity in first position. I therefore generated data with six paradigms (transitive chains, multi-hop deduction, word problems on age/height/savings/weight/speed, conversational claims, negated premises, and *indeterminate* queries whose two entities lie in disconnected premise subgraphs), with premises shuffled, distractor sentences injected, and 85% of questions balanced across both polarities, so that echoing the question does not work. Sizes: 20,000 train / 1,000 val / 2,000 test.

The main safeguard against memorisation is that training uses 20 entity names per language and testing uses a strictly disjoint pool of 15 others. No test entity (and no test combination) was ever seen during training. A 5% negation curriculum inside the training pool was added after models trained without it scored exactly 0% on negative-polarity queries.

4.2 Fine-tuning setup

Direct SFT maps the prompt straight to the final answer sentence. CoT SFT first emits a short scratchpad wrapped in special tokens (`<COT_START>` अमित `<REL_GT>` सुमित `<REL_GT>` राहुल `<COT_END>` अमित की लंबाई राहुल की लंबाई से अधिक है।), with `<REL_GT>`, `<REL_LT>`, `<REL_EQ>` and `<REL_DISJOINT>` registered as extra vocabulary items. Loss is applied only on target tokens (prompt masked with `-100`):

$$\mathcal{L}_{\text{SFT}} = - \frac{1}{\sum_t \mathbb{I}[t \geq T_{\text{prompt}}]} \sum_{t \geq T_{\text{prompt}}} \log P(x_t | x_{<t}).$$

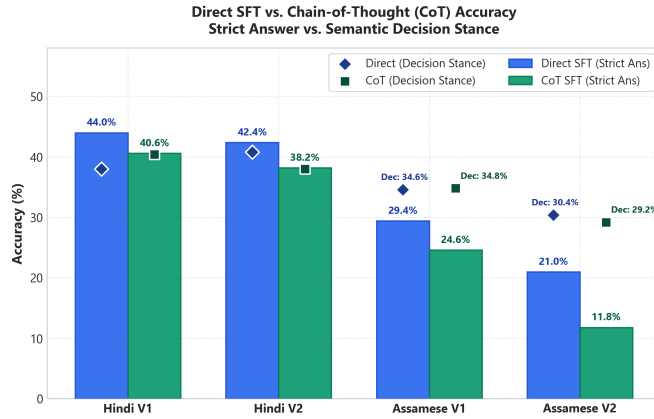
Both modes: effective batch 32, sequence length 128, up to 750 steps with early stopping. V1 used $\text{lr } 3 \times 10^{-5}$ (wd 0.1); V2 used 5×10^{-5} with decoupled wd 0.01 to protect the SwiGLU gates. Decoding is greedy with repetition penalty 1.15 and a small logit boost on prompt entity tokens so names stay grounded.

4.3 Multi-tier evaluation

Strict exact match is harsh for morphologically rich languages: a perfectly correct deduction with `-তকৈ` instead of `-তকৈ` would score zero. Loose metrics, meanwhile, can reward echoing. I therefore report five tiers: (1) strict exact match, (2) decision accuracy, the parsed relational stance (greater/less/equal/undetermined), (3) formal graph validity, the CoT steps are parsed into a graph and a solver verifies the conclusion actually follows, (4) token F1 and Levenshtein-normalised character similarity, and (5) a decomposed score weighting scratchpad F1 at 0.4 and answer F1 at 0.6.

Table 4: Phase 3 main matrix, 500 held-out queries per model (test entities fully disjoint from training). N/A: direct-SFT models emit no scratchpad.

Model	Arch	EM (ans)	Stance	Graph	F1 (ans)	Char sim	Decomp
Hindi V1 Direct	V1	44.0	38.0	N/A	83.3	85.1	N/A
Hindi V1 CoT	V1	40.6	40.4	9.4	81.9	83.7	50.8
Hindi V2 Direct	V2	42.4	40.8	N/A	82.3	84.2	N/A
Hindi V2 CoT	V2	38.2	38.0	6.4	82.0	84.5	51.6
Assamese V1 Direct	V1	29.4	34.6	N/A	66.8	75.4	N/A
Assamese V1 CoT	V1	24.6	34.8	1.8	65.2	72.9	39.5
Assamese V2 Direct	V2	21.0	30.4	N/A	56.3	66.4	N/A
Assamese V2 CoT	V2	11.8	29.2	3.0	50.7	59.3	30.7

**Figure 4:** Strict answer accuracy, all eight models.

A few patterns stand out in Table 4:

- Direct vs. CoT is a trade-off rather than a clear win for either side. Direct SFT keeps a small strict-EM edge (it only has to produce one sentence), while CoT loses some strictness but produces *verifiable* reasoning: graph validity is only defined for CoT, and CoT matches direct models on decision accuracy while also emitting the scratchpad.
- The resource gap persists: Assamese trails Hindi on every tier (e.g. 29.4% vs 44.0% best strict EM), mirroring the Phase 2 perplexity gap.
- Continuous metrics hold up better than exact match. F1 and character similarity stay high even when EM is low, which suggests that models usually name the right entity and relation and slip on inflection or template wording.

Table 5 breaks strict answer accuracy down by reasoning paradigm for all four models in each language.

Hindi	V1 Direct	V1 CoT	V2 Direct	V2 CoT
Transitive chain	56.4	47.4	47.4	39.7
Word problem	55.4	51.8	51.8	42.2
Negated relational	50.6	50.6	55.4	53.0
Multi-hop	47.9	43.6	43.6	44.7
Conversational	48.9	45.5	51.1	44.3
Indeterminate	0.0	0.0	0.0	0.0
Assamese	V1 Direct	V1 CoT	V2 Direct	V2 CoT
Multi-hop	36.2	36.2	21.3	8.5
Negated relational	36.1	28.9	24.1	8.4
Conversational	34.1	30.7	28.4	14.8
Word problem	32.5	28.9	27.7	20.5
Transitive chain	33.3	17.9	21.8	14.1
Indeterminate	0.0	0.0	0.0	4.1

Table 5: Strict answer accuracy per paradigm. “Indeterminate” means the correct answer is that the query cannot be decided.

The indeterminate row deserves a closer look. Both languages score near zero on strict EM because the gold answer is a fixed refusal template (दिए गए कथनों से यह निर्धारित नहीं किया जा सकता। / দিয়া তথ্যৰ পৰা এইটো নিৰ্ধাৰণ কৰিব নোৱাৰি). Under the looser decision metric some models do begin to answer “undetermined” correctly (up to 36.5% stance in Assamese V2 Direct). CoT models are actually *worse* here: once `<COT_START>` is emitted, the training prior demands an inequality chain, and the model fabricates a bridge (e.g. `কিশোৰ <REL_GT> কিশোৰ`) instead of emitting `<REL_DISJOINT>`. Recognising that *no path exists* is a meta-cognitive operation that 25M parameters barely have room for, so CoT acts as a “reasoning hammer” that keeps looking for a transitive path even when there is none. This is a capacity limit rather than a data problem.

4.4 Attention before and after fine-tuning

Following Section 3.2 of the assignment, I compared the pretrained and CoT-finetuned checkpoints (Modern V2, both languages). A single heatmap of one head on one sentence cannot separate a real change from noise, so the comparison is averaged over 200 held-out test examples per language, all 8 layers and all 6 heads. Each input is the prompt followed by the gold CoT completion, as in fine-tuning. Because the loss is applied only to completion tokens, I report *prompt rows* (query token inside the prompt) and *answer rows* (query token inside the completion) separately. The change is measured as the Jensen–Shannon (JS) divergence between the pretrained and finetuned attention distributions of the same query token (0 = identical, maximum $\ln 2 \approx 0.693$ nats).

Table 6: JS divergence between pretrained and finetuned attention (nats), mean over 200 test examples and 6 heads.

	Layer	0	1	2	3	4	5	6	7
Hindi	Prompt rows	0.024	0.024	0.027	0.035	0.044	0.052	0.101	0.127
	Answer rows	0.029	0.034	0.064	0.052	0.060	0.082	0.159	0.280
Assamese	Prompt rows	0.014	0.014	0.034	0.039	0.070	0.074	0.099	0.138
	Answer rows	0.023	0.024	0.047	0.062	0.107	0.130	0.140	0.267

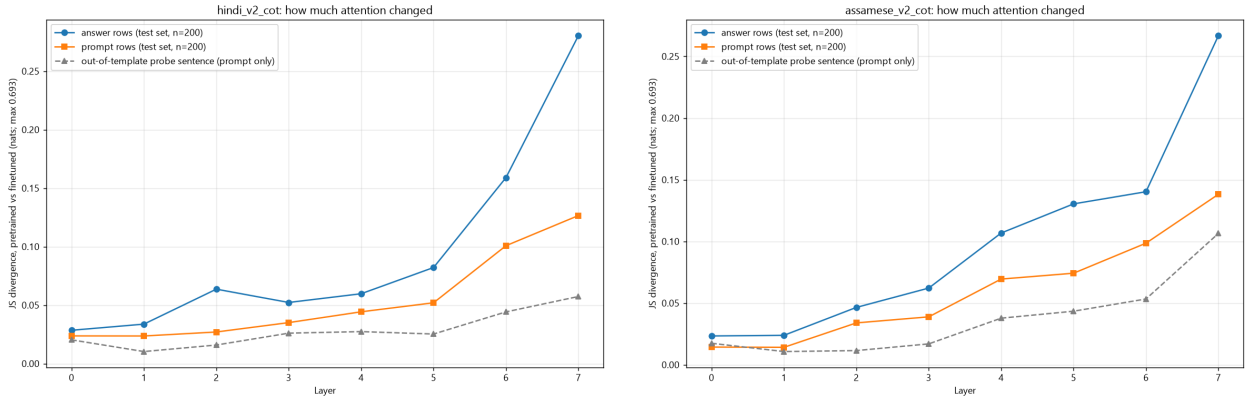


Figure 5: JS divergence by layer for Hindi (left) and Assamese (right). The grey curve is a single prompt-only sentence outside the training templates; it understates the change seen on real test examples.

Table 7: Answer-row attention statistics, pretrained $\rightarrow$ finetuned. “To prompt” is the share of attention mass that lands on prompt tokens.

Layer	Hindi			Assamese		
	Δ entropy	Δ mean dist	To prompt	Δ entropy	Δ mean dist	To prompt
0	+2.3%	−2.6%	0.56 $\rightarrow$ 0.54	+8.0%	−1.0%	0.63 $\rightarrow$ 0.62
2	−2.1%	−5.6%	0.66 $\rightarrow$ 0.61	−2.9%	−2.8%	0.63 $\rightarrow$ 0.62
5	−11.5%	−8.2%	0.38 $\rightarrow$ 0.34	−12.4%	+1.5%	0.57 $\rightarrow$ 0.60
6	−5.0%	+8.9%	0.49 $\rightarrow$ 0.59	+3.3%	+ 38.6%	0.32 $\rightarrow$ 0.44
7	− 27.1%	−1.0%	0.69 $\rightarrow$ 0.82	− 22.4%	+ 18.2%	0.56 $\rightarrow$ 0.67

Figures 6–10 show one early and one late layer per model on the first test example of each language (an indeterminate item), using the head that changed most on answer rows. Red lines mark the prompt/answer boundary: rows below the horizontal line are answer tokens, columns left of the vertical line are prompt tokens.

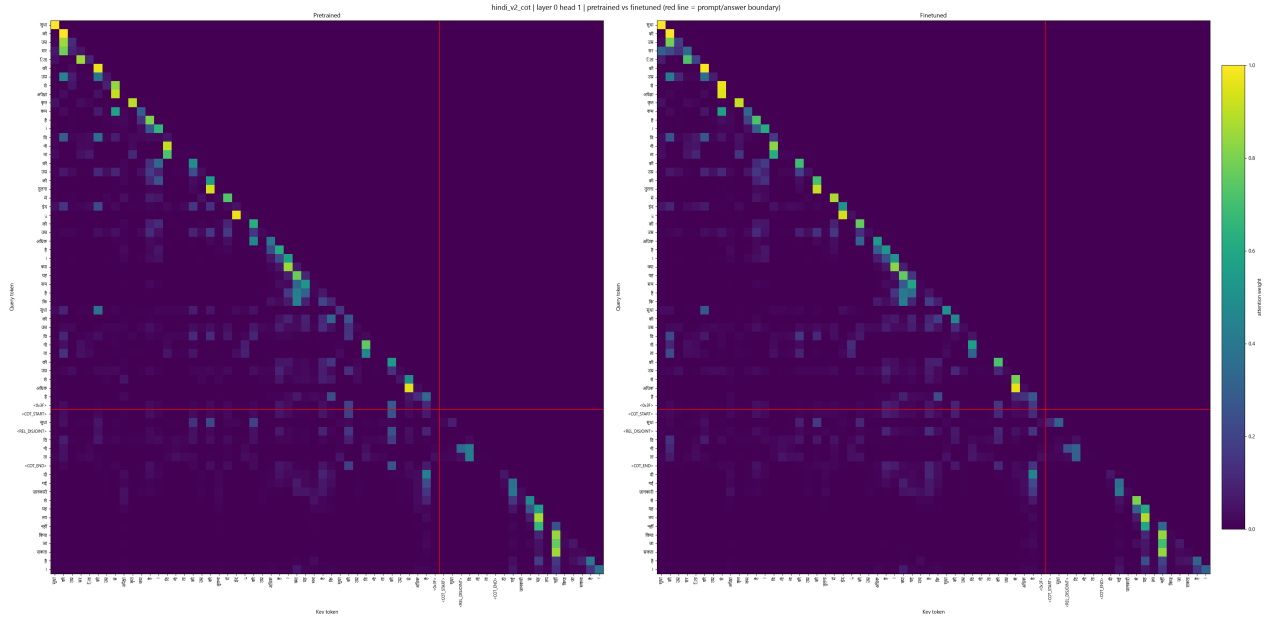


Figure 6: Hindi V2, early layer (layer 0, head 1): pretrained (left) vs. finetuned (right).

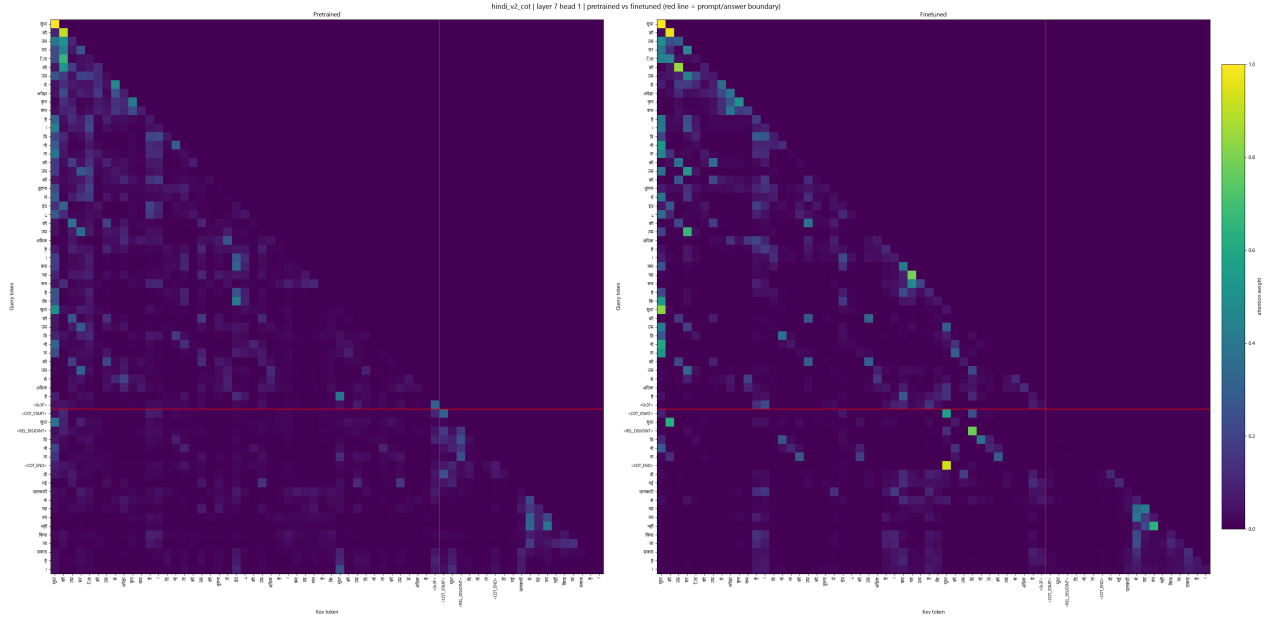


Figure 7: Hindi V2, late layer (layer 7, head 1): pretrained (left) vs. finetuned (right).

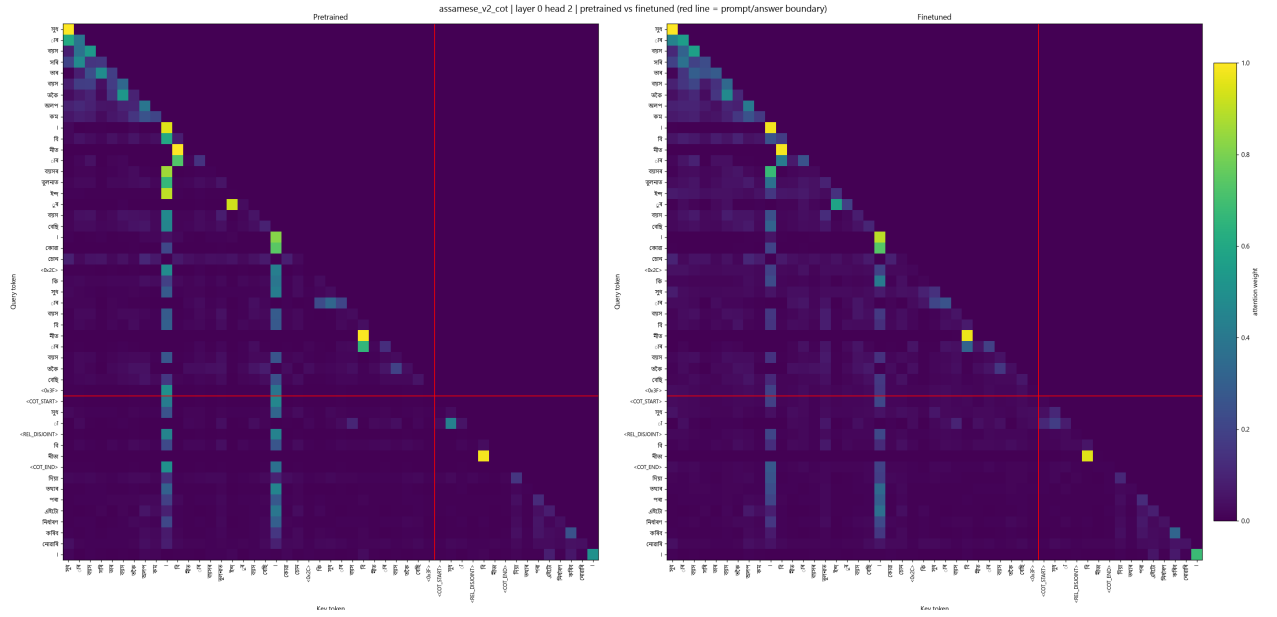


Figure 8: Assamese V2, early layer (layer 0, head 2): pretrained (left) vs. finetuned (right).

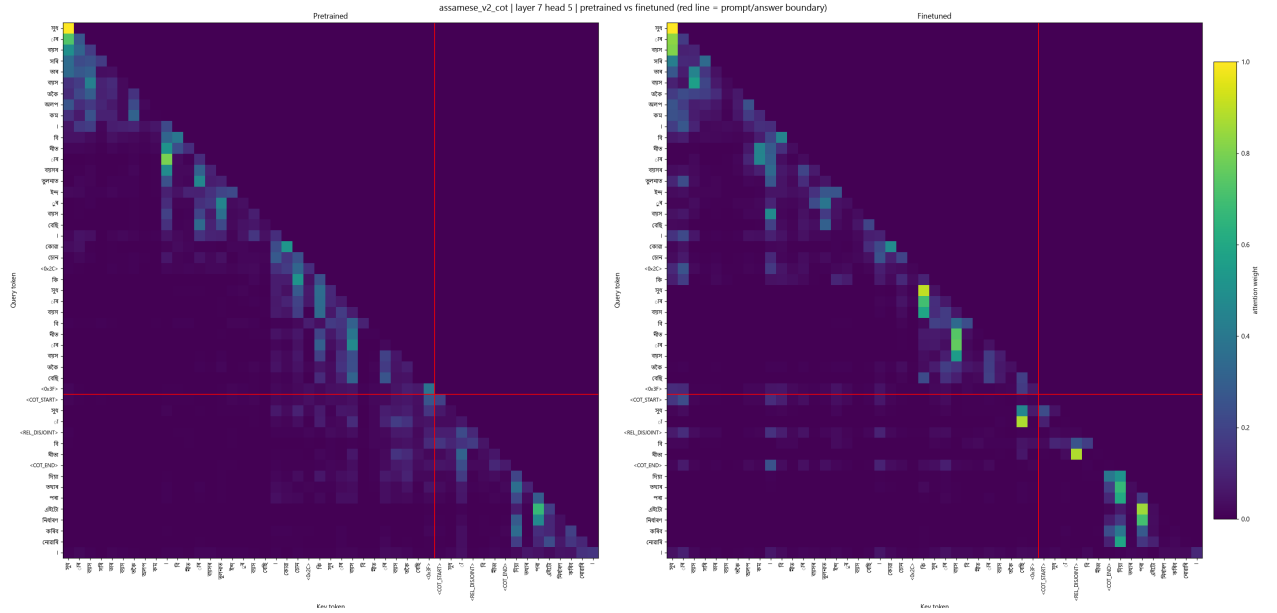


Figure 9: Assamese V2, late layer (layer 7, head 5): pretrained (left) vs. finetuned (right).

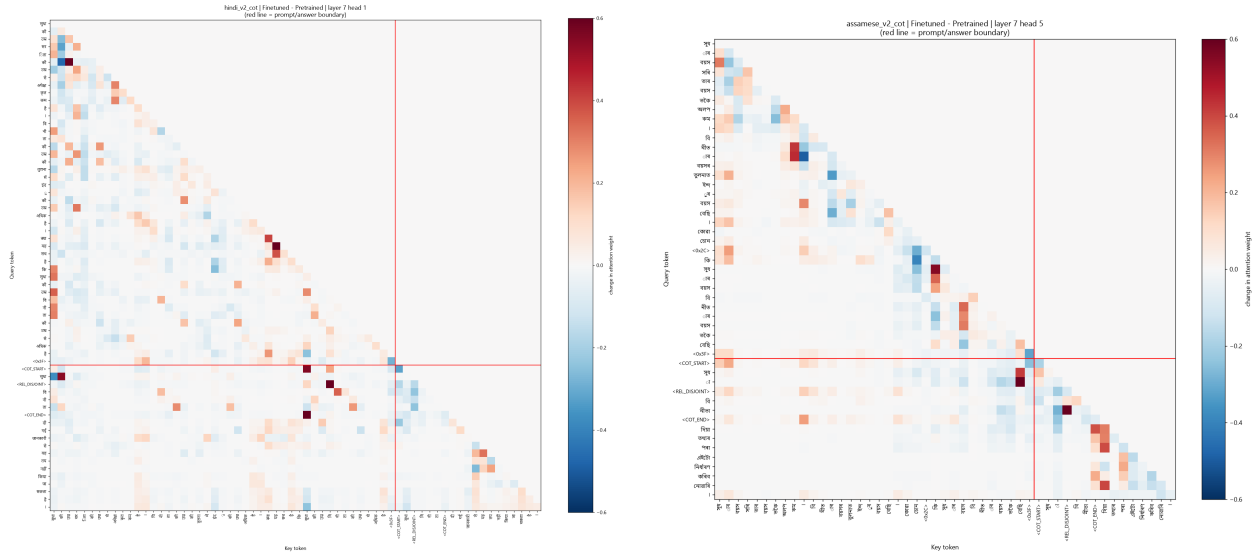


Figure 10: Finetuned minus pretrained attention for the layer-7 heads above (Hindi left, Assamese right). Red = attention gained, blue = attention lost.

The pattern is the same in both languages. *Early layers are preserved:* layers 0–2 change by at most 0.064 nats, so fine-tuning leaves the local, lexical attention learned in pretraining intact. *The last two layers carry the change, and it is largest on answer tokens* (0.280 and 0.267 at layer 7, against 0.127 and 0.138 on prompt rows), which matches a loss applied only to the completion. *Late-layer heads become sharper:* layer-7 entropy on answer rows drops by 27.1% in Hindi and 22.4% in Assamese. *Answer tokens look back at the premises more:* the share of layer-7 attention landing on the prompt rises from 0.69 to 0.82 (Hindi) and from 0.56 to 0.67 (Assamese). On local vs. long-range attention the two languages differ: in Assamese the mean attention distance grows at layers 6 and 7 (+38.6%, +18.2%), while in Hindi it is flat at layer 7 (−1.0%), so the Hindi change is a concentration of existing long-range attention onto specific premise tokens, not a lengthening.

Three limits apply. The pretrained model has never been trained on <COT_START> or the <REL_*> tokens, so part of the answer-row divergence reflects unfamiliar input; the prompt-row numbers do not have this confound and still show a clear late-layer change. Attention is measured with the gold completion fed in, not the model’s own output. And these statistics describe where attention moved; they do not show that a particular head implements a particular reasoning step.

5 The four comparison questions

This section answers the four comparison questions from the assignment brief, using the evidence from the three phases above.

5.1 Q1. How did data scale and quality differ between Model H and Model L?

Scale: Hindi ended at 723.32M tokens against Assamese’s 528.50M, a 27% gap, with both languages above the 500M target. The gap comes from supply rather than effort: large Hindi web corpora exist off the shelf, while Assamese needed targeted crawling of regional news and literary sites plus OCR of 1,840 school textbooks to reach the manual quota. *Quality:* Assamese needed more aggressive cleaning per useful token because of heavier code-switching with Bengali and English, mislabelled documents, and conjunct-glyph and spelling variation; this shows up as extra work in the script-ratio filter and the dedup stages. The end products are comparable in cleanliness (0.0% unk, deterministic splits), but Hindi’s corpus is bigger and more varied in register; Assamese’s is smaller, more textbook- and news-heavy.

5.2 Q2. How do language-modelling and reasoning results compare across the two tiers?

On intrinsic language modelling, Hindi is better on token-level metrics (test PPL 52.26 vs 80.93 for V2) but the two tiers are close at the byte level (BPB 0.5189 vs 0.5049, with Assamese actually compressing better). Part of the token-level gap is arithmetic: at fertility 1.44 vs 1.19, each Assamese word is cut into more pieces and each piece is individually less predictable. Fertility also shrinks the effective context: the same 512-token window covers roughly 430 Hindi words but only ~355 Assamese words.

Reasoning results follow the same ordering with a similar nuance. Best strict EM is 44.0% (Hindi V1 Direct) vs 29.4% (Assamese V1 Direct). On stance and continuous metrics the gap narrows: Assamese V1 CoT decision accuracy is 34.8% vs Hindi’s 40.4%, and the negation curriculum lifts Assamese negated-relation stance to 48.2% (V1 CoT), above the corresponding Hindi number. In other words, the lower-resource model loses more on surface form than on the logic

itself.

5.3 Q3. Which tokenizer / corpus factors most affected the lower-resource model?

1. **Fertility.** 1.4426 vs 1.1858 tokens/word means every Assamese sentence costs $\sim 22\%$ more context positions. For multi-hop prompts this directly shortens how much premise material fits in the window, and it inflates token-level PPL.
2. **Conjunct fragmentation.** Eastern Nagari conjuncts (ক্ষ, ভ, ত্ত) and inflectional suffixes split into multiple pieces, so a single entity occupies several attention positions (e.g. বিনীতৰ is three pieces in Figure 9).
3. **Vocabulary size choice.** A 32K vocab would have compressed Assamese better (1.31 fertility), but at 25M parameters the embedding cost would have eaten two transformer layers for both languages. 16K was the right trade-off for *model* quality even though it is not optimal for Assamese *tokenisation* in isolation.
4. **Corpus register.** The textbook-heavy Assamese manual share provided clean, consistent case morphology (-ৰ, -ক, -তকৈ), which suits the Phase 3 templates, but the overall register mix is thinner than Hindi’s news, literature and web text.

5.4 Q4. What evidence explains the observed differences?

- The fine-tuned scores reflect learning rather than memorisation. Zero-shot (pretrained, no SFT), all eight models score 0.0–0.2% strict EM on the reasoning test (16–34% stance, barely above chance), so the Phase 3 accuracies come from SFT on fully disjoint entities.
- The attention study shows where fine-tuning acted. Averaged over 200 test examples, early layers are almost unchanged, while at layer 7 the answer tokens attend 22–27% more sharply (entropy) and place more of their attention on the premises (0.69 $\rightarrow$ 0.82 in Hindi, 0.56 $\rightarrow$ 0.67 in Assamese). This is consistent with a model that learned to consult the premises while writing its rationale, and it shows up in both languages; it is descriptive evidence, not a causal test (Section 4.4).
- The failure modes are systematic. Greedy generation collapse (rep-3 75–77%) in Phase 2 and the CoT “compulsion to reason” on indeterminate queries in Phase 3 have the same root cause: a small model’s strong training prior overriding the actual input. CoT helps when a chain exists and hurts when the correct answer is to stop.
- Capacity is the main constraint. Where the two languages behave alike (architecture deltas, temperature behaviour, negation curriculum effects) the explanation is general; where they differ (fertility, conjunct splitting, the EM gap) the explanation traces back to tokenisation and corpus supply, since both pipelines are otherwise identical.